A Source Quine in the Derived Operator–MF Residual

The `nonsofic_existence` formalization

August 25, 2026

Abstract

We construct one fixed finitely presented non-MF group U containing an exact encoding of this manuscript, including the executable program printed below. The encoding is internal: for every source position i and proposed cell b, a word $W(i,b)$ belongs to $[\mathrm{Rad}_{\mathrm{MF}}(U), U]$ and

$$W(i,b) = \mathbf{1} \quad \Longleftrightarrow \quad b \text{ is the source cell at position } i.$$

The cells are the file's literal bytes, followed by the sentinel 256. Thus the identity pattern of words lying in the operator–MF invisible part recovers the complete file, not merely a hash or a semantic description. The displayed Python program is a strict source quine: its standard output is byte-for-byte this TeX file. A Lean theorem proves the group-theoretic and computability layers; an independent checker executes the quine, compares bytes, and compares the generated Lean byte ledger with this file.

1 The fixed invisible computer

For a dimension sequence $\mathbf{d} = (d_n)$, put

$$\mathcal{Q}_{\mathbf{d}} = \prod_n M_{d_n}(\mathbb{C}) / \bigoplus_n M_{d_n}(\mathbb{C}),$$

where the ideal is norm-null. For a group G, define

$$\mathrm{Rad}_{\mathrm{MF}}(G) = \bigcap_{\mathbf{d}} \bigcap_{\rho : G \to U(\mathcal{Q}_{\mathbf{d}})} \ker \rho.$$

An element is called operator–MF invisible when it belongs to this intersection.

The formal development supplies a finitely presented group E and an element $z \in E$ such that

$$z \neq \mathbf{1}, \qquad z \in \mathrm{Rad}_{\mathrm{MF}}(E), \qquad E \text{ is not operator–MF.}$$

It also supplies one fixed finitely presented Boone controller B. For every partial-recursive program code c, there is an explicit controller word $v_c \in B$ satisfying

$$v_c = \mathbf{1} \quad \Longleftrightarrow \quad c(0) \text{ is defined.}$$

These are closed theorems in the repository, not hypotheses of the result in this paper.

Define the fixed carrier

$$U = E * B$$

and the faithful radical gate

$$R_c = [z, v_c] \in U.$$

Here the letters denote their canonical free-factor images. Free-product normal forms give

$$[z, v_c] = \mathbf{1} \quad \Longleftrightarrow \quad v_c = \mathbf{1},$$

because $z \neq \mathbf{1}$ and the second input belongs to the other free factor. Functoriality of the MF radical gives $z \in \mathrm{Rad}_{\mathrm{MF}}(U)$, hence

$$R_c \in [\mathrm{Rad}_{\mathrm{MF}}(U), U] \subseteq \mathrm{Rad}_{\mathrm{MF}}(U).$$

The same commutator is therefore simultaneously invisible and a faithful readout of the controller. This coupling is the point: the computation is not stored in an unrelated direct factor or in an external label.

The group U is finitely presented because both factors are. It is non-MF because it contains the non-MF factor E and operator–MF is inherited by subgroups.

2 Output-sensitive words

The domain detector R_c alone records only a halting bit. To encode source data, fix a program q, a position i, and a proposed natural-number cell b. Let $t(q, i, b)$ be a partial-recursive program which runs $q(i)$ and halts precisely when the returned value equals b. Define

$$W_q(i, b) = R_{t(q,i,b)}.$$

Output-query theorem. For every q, i, b,

$$W_q(i, b) \in [\mathrm{Rad}_{\mathrm{MF}}(U), U] \quad \text{and} \quad W_q(i, b) = \mathbf{1} \Longleftrightarrow b \in \mathrm{eval}(q, i).$$

The notation on the right is membership in a partial value: it means exactly that $q(i)$ terminates with output b.

The proof is literal. The filter $t(q, i, b)$ is obtained by composing the universal evaluator with a partial equality acceptor. The Boone compiler turns its domain into $v_{t(q,i,b)} = \mathbf{1}$. The faithful radical gate preserves that equality truth value while placing the resulting word in the derived MF residual.

If q computes a total stream $s : \mathbb{N} \to \mathbb{N}$, then

$$W_q(i, b) = \mathbf{1} \Longleftrightarrow s(i) = b. \tag{1}$$

For every wrong candidate $b \neq s(i)$, the word $W_q(i, b)$ is therefore a *nonidentity* element of $\mathrm{Rad}_{\mathrm{MF}}(U)$. Equation (1) is an exact equality oracle, not an assertion that all encoding words collapse to the identity.

3 The exact source quine

The following Python 3 program is part of this TeX source. Its first assignment contains a quoted template, split into bounded physical lines, with one distinguished token. Replacing that token by the same deterministic chunked representation reconstructs the program inside the same surrounding manuscript.

```
TEMPLATE = (
    '\\documentclass[11pt]{article}\n\\usepackage[T1]{fontenc}\n\\usepackage{amsma'
    'th,amssymb,mathtools}\n\\usepackage[a4paper,margin=30mm]{geometry}\n\\usepac'
    'kage[colorlinks=true,linkcolor=blue,urlcolor=blue]{hyperref}\n\\usepackage'
    '{listings}\n\\usepackage{microtype}\n\n\\lstset{\n  basicstyle=\\ttfamily\\scrip'
```

```
'tsize,\n  breaklines=true,\n  breakatwhitespace=false,\n  columns=fullflexi'
'ble,\n  keepspaces=true,\n  showstringspaces=false\n}\n\n\\newcommand{\\RadMF}{'
'\\operatorname{Rad}_{\\mathrm{MF}}}\n\\newcommand{\\Qd}{\\mathcal Q_{\\mathbf d'
'}}\n\\newcommand{\\one}{\\mathbf 1}\n\n\\title{A Source Quine in the Derived Op'
'erator--MF Residual}\n\\author{The \\texttt{nonsofic\\_existence} formalizat'
'ion}\n\\date{August 25, 2026}\n\n\\begin{document}\n\\maketitle\n\n\\begin{abstrac'
't}\nWe construct one fixed finitely presented non-MF group $U$ containing'
' an\nexact encoding of this manuscript, including the executable program '
'printed\nbelow.  The encoding is internal: for every source position $i$ '
'and proposed\ncell $b$, a word $W(i,b)$ belongs to\n$[\\RadMF(U),U]$ and\n\\['
'\n  W(i,b)=\\one \\quad\\Longleftrightarrow\\quad\n  b\\text{ is the source cel'
"l at position }i.\n\\]\nThe cells are the file's literal bytes, followed by"
' the sentinel $256$.\nThus the identity pattern of words lying in the ope'
'rator--MF invisible part\nrecovers the complete file, not merely a hash o'
'r a semantic description.\nThe displayed Python program is a strict sourc'
'e quine: its standard output is\nbyte-for-byte this TeX file.  A Lean the'
'orem proves the group-theoretic and\ncomputability layers; an independent'
' checker executes the quine, compares\nbytes, and compares the generated '
'Lean byte ledger with this file.\n\\end{abstract}\n\n\\section{The fixed invi'
'sible computer}\n\nFor a dimension sequence $\\mathbf d=(d_n)$, put\n\\[\n \\Qd'
'=\\prod_n M_{d_n}(\\mathbb C)\\big/\\bigoplus_n M_{d_n}(\\mathbb C),\n\\]\nwhere'
' the ideal is norm-null.  For a group $G$, define\n\\[\n \\RadMF(G)=\\bigcap_'
'{\\mathbf d}\\ \\bigcap_{\\rho:G\\to U(\\Qd)}\\ker\\rho.\n\\]\nAn element is called'
' operator--MF invisible when it belongs to this\nintersection.\n\nThe forma'
'l development supplies a finitely presented group $E$ and an element\n$z\\'
'in E$ such that\n\\[\n z\\ne\\one,\\qquad z\\in\\RadMF(E),\\qquad E\\text{ is not '
'operator--MF}.\n\\]\nIt also supplies one fixed finitely presented Boone co'
'ntroller $B$.  For every\npartial-recursive program code $c$, there is an'
' explicit controller word\n$v_c\\in B$ satisfying\n\\[\n v_c=\\one\\quad\\Longle'
'ftrightarrow\\quad c(0)\\text{ is defined}.\n\\]\nThese are closed theorems i'
'n the repository, not hypotheses of the result in\nthis paper.\n\nDefine th'
'e fixed carrier\n\\[\n U=E*B\n\\]\nand the faithful radical gate\n\\[\n R_c=[z,v_'
'c]\\in U.\n\\]\nHere the letters denote their canonical free-factor images. '
' Free-product\nnormal forms give\n\\[\n [z,v_c]=\\one\\quad\\Longleftrightarrow'
'\\quad v_c=\\one,\n\\]\nbecause $z\\ne\\one$ and the second input belongs to th'
'e other free factor.\nFunctoriality of the MF radical gives $z\\in\\RadMF(U'
')$, hence\n\\[\n R_c\\in[\\RadMF(U),U]\\subseteq\\RadMF(U).\n\\]\nThe same commuta'
'tor is therefore simultaneously invisible and a faithful\nreadout of the '
'controller.  This coupling is the point: the computation is\nnot stored i'
'n an unrelated direct factor or in an external label.\n\nThe group $U$ is '
'finitely presented because both factors are.  It is non-MF\nbecause it co'
'ntains the non-MF factor $E$ and operator--MF is inherited by\nsubgroups.'
'\n\n\\section{Output-sensitive words}\n\nThe domain detector $R_c$ alone reco'
'rds only a halting bit.  To encode source\ndata, fix a program $q$, a pos'
'ition $i$, and a proposed natural-number cell\n$b$.  Let $t(q,i,b)$ be a '
'partial-recursive program which runs $q(i)$ and\nhalts precisely when the'
' returned value equals $b$.  Define\n\\[\n W_q(i,b)=R_{t(q,i,b)}.\n\\]\n\n\\para'
'graph{Output-query theorem.}\nFor every $q,i,b$,\n\\[\n W_q(i,b)\\in[\\RadMF(U'
'),U]\n \\quad\\text{and}\\quad\n W_q(i,b)=\\one\\Longleftrightarrow b\\in\\operat'
'orname{eval}(q,i).\n\\]\nThe notation on the right is membership in a parti'
'al value: it means exactly\nthat $q(i)$ terminates with output $b$.\n\nThe '
'proof is literal.  The filter $t(q,i,b)$ is obtained by composing the\nun'
'iversal evaluator with a partial equality acceptor.  The Boone compiler\n'
'turns its domain into $v_{t(q,i,b)}=\\one$.  The faithful radical gate pr'
'eserves\nthat equality truth value while placing the resulting word in th'
'e derived\nMF residual.\n\nIf $q$ computes a total stream $s:\\mathbb N\\to\\m'
'athbb N$, then\n\\[\n W_q(i,b)=\\one\\Longleftrightarrow s(i)=b.                '
'    \\tag{1}\n\\]\nFor every wrong candidate $b\\ne s(i)$, the word $W_q(i,b)'
'$ is therefore a\n\\emph{nonidentity} element of $\\RadMF(U)$.  Equation (1'
') is an exact equality\noracle, not an assertion that all encoding words '
'collapse to the identity.\n\n\\section{The exact source quine}\n\nThe followi'
'ng Python 3 program is part of this TeX source.  Its first\nassignment co'
'ntains a quoted template, split into bounded physical lines, with\none di'
'stinguished token.  Replacing that token by the same deterministic\nchunk'
'ed representation reconstructs the program inside the same surrounding\nm'
'anuscript.\n\n% QUINE-PROGRAM-BEGIN\n\\begin{lstlisting}[language=Python]\nTE'
'MPLATE = @@TEMPLATE_REPR@@\ndef quote_chunks(text, width=72):\n    return '
'"(\\n" + "".join(\n        f"    {text[index:index + width]!r}\\n"\n        '
'for index in range(0, len(text), width)\n    ) + ")"\nprint(TEMPLATE.repla'
'ce("@@TEMPLATE_REPR@@", quote_chunks(TEMPLATE), 1), end="")\n\\end{lstlist'
```

```
’ing}\n% QUINE-PROGRAM-END\n\nThis is a source quine in the strict sense use’
’d here:\n\\[\n \\operatorname{stdout}(\\text{program above})\n   =\\text{the co’
’mplete byte sequence of this file}.       \\tag{2}\n\\]\nNo file is read, no’
’ network or environment input is consulted, and no\nmanuscript payload is’
" appended by a wrapper.  The quoted template is the\nprogram’s own data, "
"while the replacement operation installs that data’s\nrepresentation at t"
’he unique self-reference site.  Equation (2) is checked\nby executing the’
’ extracted program in an empty temporary working directory\nand performin’
’g a byte comparison.\n\n\\section{Putting the whole manuscript in the invis’
’ible part}\n\nLet $M$ be the literal byte list of this file and let $N=|M|’
’$.  Define the\ntotal source stream\n\\[\n s_M(i)=\n \\begin{cases}\n   M_i,&i<’
’N,\\\\\n   256,&i\\ge N.\n \\end{cases}\n\\]\nAll file cells lie in $\\{0,\\ldots,2’
’55\\}$, so $256$ is an unambiguous end\nsentinel.  A finite-list lookup is’
’ computable.  Universality of the\npartial-recursive code language theref’
’ore yields a code $q_M$ with\n\\[\n \\operatorname{eval}(q_M,i)=s_M(i)\\qquad’
’(i\\in\\mathbb N).\n\\]\nApply the output-query theorem to $q_M$ and set\n\\[\n ’
’\\mathcal W_M=\\{W_{q_M}(i,b):i\\in\\mathbb N, 0\\le b\\le256\\}.\n\\]\nEvery memb’
’er of $\\mathcal W_M$ lies in $[\\RadMF(U),U]$.  Starting at $i=0$,\nthe un’
’ique $b\\le256$ for which $W_{q_M}(i,b)=\\one$ is $s_M(i)$.  Emit it when\n’
’$b<256$ and stop when $b=256$.  This decoder recovers exactly $M$.\n\nComb’
’ining this with (2), the decoded object is simultaneously the complete\nm’
’anuscript and an executable program which outputs that same complete\nman’
’uscript.  In particular, the invisible encoding includes every byte of t’
"he\nprogram’s quoted template, both lines of its evaluator, every theorem"
’\nstatement here, and the decoder description itself.\n\n\\paragraph{Finite ’
’realization.}\nAlthough $\\mathcal W_M$ is written as a total stream inter’
’face, the file is\nfinite.  Its complete content is determined by the fin’
’ite family\n\\[\n \\{W_{q_M}(i,b):0\\le i\\le N,\\ 0\\le b\\le256\\}.\n\\]\nAt $i=N$ ’
’the unique identity answer is the sentinel.  The infinite extension\nonly’
’ makes the source program total and does not hide any infinitary group\np’
’resentation: $U$ itself remains one fixed finite presentation.\n\n\\section’
’{Why this is not a wrapper}\n\nThere are three distinct statements, and no’
’ne is substituted for another.\n\nFirst, the Python fixed point is syntact’
’ic and exact.  The checker compares\nits output with the source bytes, no’
’t with a hash and not with a normalized\nversion of the TeX.\n\nSecond, tho’
’se exact bytes are the data of a computable stream compiled into\nthe Boo’
’ne factor.  The observable words are not the controller words alone:\nthe’
’y are commutators with the nonidentity radical mark.  The free-product g’
’ate\nproves that this mixing preserves the output truth table exactly, wh’
’ile\nradical functoriality proves that every mixed word is invisible to e’
’very\nmatrix-corona representation.  Wrong byte guesses give nontrivial i’
’nvisible\nelements, so the encoding has genuine internal contrast.\n\nThird’
’, self-reference is not advertised as the analytic source of non-MF.\nThe’
’ carrier depends on the independently proved non-MF seed $E$.  The quine’
’\nadds exact computation and self-description inside its MF residual; it ’
’does\nnot reprove the seed obstruction.  This separation avoids the circu’
’lar claim\nthat a manuscript declaring its group non-MF somehow makes the’
’ group non-MF.\n\nThus the construction is stronger than juxtaposing a non’
’-MF group with a\nquine, but narrower than a self-certifying analytic obs’
’truction.  Its precise\ncontent is an exact source fixed point whose comp’
’lete equality oracle lives in\nthe derived MF-invisible subgroup of a clo’
’sed, finitely presented non-MF\ncarrier.\n\n\\section{Formal and executable ’
’custody}\n\nThe Lean module\n\\texttt{GroupApproximation.Computability.MFRad’
’icalQuine} proves:\n\\begin{itemize}\n\\item partial recursiveness of the ou’
’tput equality filter;\n\\item $W_q(i,b)\\in[\\RadMF(U),U]$ and operator--MF ’
’invisibility;\n\\item $W_q(i,b)=\\one$ exactly when $q(i)$ returns $b$;\n\\it’
’em existence of an invisible source encoding for every computable stream’
’.\n\\end{itemize}\n\nThe generated module\n\\texttt{GroupApproximation.Computa’
’bility.MFRadicalQuineSource} contains the\nliteral natural-number list of’
" this file’s bytes, defines the sentinel stream,\nproves that stream comp"
’utable, and instantiates the invisible-source theorem.\nThe list is gener’
’ated only after the TeX fixed point is complete, so it is a\ncustody ledg’
’er rather than an assumption used to manufacture the quine.\n\nThe indepen’
’dent checker\n\\begin{center}\n\\path{scripts/check_mf_radical_tex_quine.py}’
’\n\\end{center}\nperforms four checks:\n\\begin{enumerate}\n\\item regenerate t’
’he fixed point from the canonical template;\n\\item extract and execute th’
’e embedded source program;\n\\item compare both results byte-for-byte with’
’ this file;\n\\item parse the Lean byte ledger and compare every integer w’
’ith the same file.\n\\end{enumerate}\nThe Lean build then checks the group-’
’theoretic theorem and the concrete source\ninstantiation with the kernel.’
```

```
    '\n\n\\section{Conclusion}\n\nOne fixed finitely presented non-MF group $U$ no'
    'w carries a complete source\nquine internally in its derived MF residual.'
    '  The construction has two exact\nfixed points: a syntactic fixed point p'
    'roducing the TeX bytes, and a faithful\ngroup-theoretic gate turning outp'
    'ut equality into identity while retaining\nMF invisibility.  Their interf'
    'ace is the literal source-cell stream, verified\nindependently and formal'
    'ized as a computable object.\n\n\\end{document}\n'
)
def quote_chunks(text, width=72):
    return "(\n" + "".join(
        f"    {text[index:index + width]!r}\n"
        for index in range(0, len(text), width)
    ) + ")"
print(TEMPLATE.replace("@@TEMPLATE_REPR@@", quote_chunks(TEMPLATE), 1), end="")
```

This is a source quine in the strict sense used here:

$$\text{stdout(program above)} = \text{the complete byte sequence of this file.} \tag{2}$$

No file is read, no network or environment input is consulted, and no manuscript payload is appended by a wrapper. The quoted template is the program's own data, while the replacement operation installs that data's representation at the unique self-reference site. Equation (2) is checked by executing the extracted program in an empty temporary working directory and performing a byte comparison.

4 Putting the whole manuscript in the invisible part

Let M be the literal byte list of this file and let $N = |M|$. Define the total source stream

$$s_M(i) = \begin{cases} M_i, & i < N, \\ 256, & i \geq N. \end{cases}$$

All file cells lie in $\{0, \ldots, 255\}$, so 256 is an unambiguous end sentinel. A finite-list lookup is computable. Universality of the partial-recursive code language therefore yields a code q_M with

$$\text{eval}(q_M, i) = s_M(i) \qquad (i \in \mathbb{N}).$$

Apply the output-query theorem to q_M and set

$$\mathcal{W}_M = \{W_{q_M}(i, b) : i \in \mathbb{N}, 0 \leq b \leq 256\}.$$

Every member of $\mathcal{W}_M$ lies in $[\text{Rad}_{\text{MF}}(U), U]$. Starting at $i = 0$, the unique $b \leq 256$ for which $W_{q_M}(i, b) = \mathbf{1}$ is $s_M(i)$. Emit it when $b < 256$ and stop when $b = 256$. This decoder recovers exactly M.

Combining this with (2), the decoded object is simultaneously the complete manuscript and an executable program which outputs that same complete manuscript. In particular, the invisible encoding includes every byte of the program's quoted template, both lines of its evaluator, every theorem statement here, and the decoder description itself.

Finite realization. Although $\mathcal{W}_M$ is written as a total stream interface, the file is finite. Its complete content is determined by the finite family

$$\{W_{q_M}(i, b) : 0 \leq i \leq N,\ 0 \leq b \leq 256\}.$$

At $i = N$ the unique identity answer is the sentinel. The infinite extension only makes the source program total and does not hide any infinitary group presentation: U itself remains one fixed finite presentation.

5 Why this is not a wrapper

There are three distinct statements, and none is substituted for another.

First, the Python fixed point is syntactic and exact. The checker compares its output with the source bytes, not with a hash and not with a normalized version of the TeX.

Second, those exact bytes are the data of a computable stream compiled into the Boone factor. The observable words are not the controller words alone: they are commutators with the nonidentity radical mark. The free-product gate proves that this mixing preserves the output truth table exactly, while radical functoriality proves that every mixed word is invisible to every matrix-corona representation. Wrong byte guesses give nontrivial invisible elements, so the encoding has genuine internal contrast.

Third, self-reference is not advertised as the analytic source of non-MF. The carrier depends on the independently proved non-MF seed E. The quine adds exact computation and self-description inside its MF residual; it does not reprove the seed obstruction. This separation avoids the circular claim that a manuscript declaring its group non-MF somehow makes the group non-MF.

Thus the construction is stronger than juxtaposing a non-MF group with a quine, but narrower than a self-certifying analytic obstruction. Its precise content is an exact source fixed point whose complete equality oracle lives in the derived MF-invisible subgroup of a closed, finitely presented non-MF carrier.

6 Formal and executable custody

The Lean module `GroupApproximation.Computability.MFRadicalQuine` proves:

- partial recursiveness of the output equality filter;
- $W_q(i,b) \in [\mathrm{Rad}_{\mathrm{MF}}(U), U]$ and operator–MF invisibility;
- $W_q(i,b) = \mathbf{1}$ exactly when $q(i)$ returns b;
- existence of an invisible source encoding for every computable stream.

The generated module `GroupApproximation.Computability.MFRadicalQuineSource` contains the literal natural-number list of this file's bytes, defines the sentinel stream, proves that stream computable, and instantiates the invisible-source theorem. The list is generated only after the TeX fixed point is complete, so it is a custody ledger rather than an assumption used to manufacture the quine.

The independent checker

`scripts/check_mf_radical_tex_quine.py`

performs four checks:

1. regenerate the fixed point from the canonical template;
2. extract and execute the embedded source program;
3. compare both results byte-for-byte with this file;
4. parse the Lean byte ledger and compare every integer with the same file.

The Lean build then checks the group-theoretic theorem and the concrete source instantiation with the kernel.

7 Conclusion

One fixed finitely presented non-MF group U now carries a complete source quine internally in its derived MF residual. The construction has two exact fixed points: a syntactic fixed point producing the TeX bytes, and a faithful group-theoretic gate turning output equality into identity while retaining MF invisibility. Their interface is the literal source-cell stream, verified independently and formalized as a computable object.